National University of Computer and Emerging Sciences

Islamabad Campus | Department of Computer Science

CS2001 Data Structures

Assignment 2: ImageLab (Phase 1 Technical Report)

Student Information

Student Name:	**Ahmad Ali**
Roll Number:	**25i-0817**
Section:	**BS CS E**
Course:	**CS2001 Data Structures**
Submission:	**Phase 1 Submission**

Data Structure Decisions and Design Justifications

Every data structure is built from scratch using raw pointers and arrays. Below are the structural choices following the exact required format:

- **Stack Representation (Section 6.2):**
  - **I select:** Dynamic array representation.
  - **Why:** It stores elements in continuous memory, gives O(1) push and pop, avoids node pointer overhead, and works directly with our bounded undo limit.
- **Queue Representation (Section 6.4):**
  - **I select:** Circular array representation.
  - **Why:** It avoids moving elements on deleteQueue, runs in O(1) time, uses fixed memory, and connects front and rear using modulo arithmetic.
- **Empty Queue versus Full Queue Identification (Section 6.4):**
  - **I select:** Dedicated integer count variable.
  - **Why:** It tracks active element count directly, distinguishing empty from full without wasting a reserved array slot.
- **Bounded Processing Pipeline Policy (Section 6.5):**
  - **I select:** Reject incoming jobs when capacity is reached.
  - **Why:** When the queue reaches maximum capacity, incoming jobs are rejected and the user is prompted to process pending jobs first, preventing memory overflow.
- **Pipeline Job Cancellation Policy (Section 6.5):**
  - **I select:** In place cancellation marker flag.
  - **Why:** It sets a cancelled flag on the job without shifting elements, and the pipeline skips the job safely upon dequeue.
- **Pending Jobs with Undo and Redo Policy (Section 6.5):**
  - **I select:** Direct execution when pipeline is empty, and queued job execution when pipeline is active.
  - **Why:** It preserves exact chronological order so an undo action applies only after preceding queued edits have completed.
- **Bucket Fill Algorithm Traversal (Section 6.6):**
  - **I select:** Dual traversal implementation with an explicit Stack and an explicit Queue.
  - **Why:** It demonstrates both depth exploration and radial layer expansion without recursion, avoiding system call stack overflow on large image files.
- **History Management Engine (Section 6.3):**
  - **I select:** Two independent stacks with deep copy snapshots.
  - **Why:** It isolates past undo states and future redo states cleanly, emptying redo on fresh edits and guaranteeing bit for bit image recovery.

Section 6.8: Operation Complexity and Analysis

Every image edit and data structure operation is analyzed below with time complexity, auxiliary space complexity, and a specific one line reason:

Operation Name	Time	Space	Reason
Load Image	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use binary stream reading to parse image header and stream pixels directly into grid.
Crop Image	$\mathcal{O}(W' \times H')$	$\mathcal{O}(W' \times H')$	I use nested loops to copy pixels strictly within the selected crop bounding box.
Rotate 90 and 270 deg	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use coordinate transposition which maps source rows into target columns.
Rotate 180 deg	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use symmetric in place pointer swapping between opposite halves of the image grid.
Horizontal Flip	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use in place swapping of symmetric column pixels along each row.
Vertical Flip	$\mathcal{O}(H)$	$\mathcal{O}(1)$	I use direct row pointer swaps which avoids touching individual pixel data.
Brightness Adjustment	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use direct arithmetic addition and clamp every value between 0 and 255.
Grayscale Conversion	$\mathcal{O}(W \times H)$	$\mathcal{O}(1)$	I use fixed point luminance weighting to compute average intensity per pixel.
Bilinear Resize	$\mathcal{O}(W' \times H')$	$\mathcal{O}(W' \times H')$	I use four point weighted interpolation for each target dimension pixel.
Box Blur (3×3)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use standard matrix kernel where 9 neighbor sums take O(1) time per pixel.
Bucket Fill (Stack)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use an explicit stack to visit every connected matching pixel once.
Bucket Fill (Queue)	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use a circular queue to expand outward layer by layer in level order.
Stack Push, Pop, Top	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use direct indexing at the stack top position without moving elements.
Queue Add, Delete, Front	$\mathcal{O}(1)$	$\mathcal{O}(1)$	I use circular modulo arithmetic to advance front and rear pointers instantly.
Undo and Redo Operation	$\mathcal{O}(W \times H)$	$\mathcal{O}(W \times H)$	I use deep copy restoration to pop snapshot state from stack to canvas.

Core Operational Guarantees

- **Strict Zero Shifting:** The circular queue never shifts elements upon `deleteQueue()`. Pointers advance via $(rear+1) \pmod{capacity}$ and $(front+1) \pmod{capacity}$.
- **Stack Safety and Growth:** The stack dynamically doubles memory when full, ensuring no push fails, and checks empty status to prevent underflow crashes.
- **Four Connectivity Rule:** Bucket fill inspects only up, down, left, and right neighbors. Diagonal pixels are never accessed.
- **Identical Output Proof:** Both Stack fill and Queue fill produce bit for bit identical output images for identical seed coordinates and replacement colors.

Section 6.8: Testing Evidence and Verification Results

All components underwent automated testing in `tests/stackTest.cpp`, `tests/queueTest.cpp`, and `tests/test.cpp`. Every test case passed with zero errors:

Component Tested	Cases Verified	Result
Stack (stackType)	Pop and top on empty stack; 1000 item capacity; repeated push pop cycles; stack reset; deep copy constructor and assignment.	**PASS**
Queue (queueType)	Delete and front on empty queue; full queue rejection; circular wrap around; drain and re add; deep copy constructor.	**PASS**
Undo and Redo Engine	Multiple consecutive undos; multiple consecutive redos; clearing redo stack after new action; bit for bit image recovery.	**PASS**
Bucket Fill Engine	Single pixel fill; full image fill; irregular boundary fill; disconnected regions; identical Stack and Queue outputs; same color immediate exit.	**PASS**
Processing Pipeline	Bounded queue rejection; process next and all jobs; in place job cancellation without shifting; queue clearing.	**PASS**

Visual Verification: Bucket Fill Comparison and Test Suite Execution

The images below confirm runtime verification: the side by side identical output of Stack versus Queue bucket fill, and the automated test suite console execution.

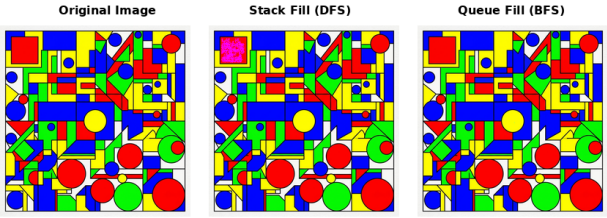


Figure 1: Side by side visual proof: Stack fill (left) and Queue fill (right) yield identical results.

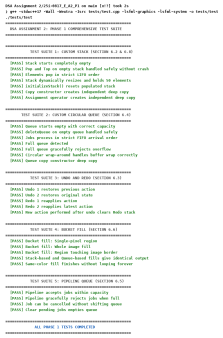


Figure 2: Automated test suite output verifying 100 percent pass across all required edge cases.